Criterion-Level Clinical-Trial Eligibility Screening with a Fine-Tuned Small Transformer: KV-Cached Decoding, Tiled Online-Softmax Attention and Speculative Decoding

Zaid Fareed
23i-0829
Department of CS, FAST-NUCES ISB
Generative AI (Semester 7), Assignment 2

Code: https://github.com/Zaid-fareed/i230829_C_GenAi-A2
Model weights (LoRA adapters):
https://huggingface.co/ZaidFareed/i230829-genai-a2-adapters
Demo video: https://youtu.be/uVTm5AuokaU

***Abstract*—Matching a patient to a clinical trial requires deciding, for every inclusion and exclusion criterion, whether the patient record satisfies it, contradicts it, or simply does not say. We build and evaluate a complete, locally executable system for this criterion-level task on the TrialGPT Criterion Annotations corpus using a patient-grouped 37/8/8 split. An open-weight decoder (SmolLM2-1.7B-Instruct on a free GPU; a 360M variant on a laptop CPU) is adapted with LoRA to emit a structured JSON answer containing one of the dataset's expert label categories and the IDs of the supporting patient sentences. On the untouched test split the adapted 1.7B model reaches 0.709 accuracy and 0.460 macro-F1 (zero-shot base model: 0.146 and 0.111; majority-label baseline: 0.626 and 0.272), with micro evidence-sentence F1 of 0.384. An error analysis shows that the gain comes from inclusion criteria, whereas for exclusion criteria the model collapses to the majority label *not excluded*. Beyond accuracy, we re-implement the inference stack from first principles: a Llama-style decoder with an explicit, rollback-capable KV cache; a tiled attention kernel with online softmax; and greedy speculative decoding with a LoRA-adapted draft model. Every mechanism is validated for exact agreement with a reference path (the cache is 18.7× faster than recomputation at 512+128 tokens; speculative decoding is exact but, honestly, slower than greedy with the chosen draft) and benchmarked for latency, throughput and memory on a GPU; a second, laptop-CPU configuration with a 360M model supplies the locally hosted application. The pieces are integrated in a browser application that exposes per-criterion evidence, a live decoding-mode comparison and exportable reports. The system is an educational prototype and not a clinical decision tool.**

***Index Terms*—clinical trial matching, eligibility criteria, LoRA, KV cache, online softmax, speculative decoding, small language models**

I. Introduction

Enrolling the right patients in a clinical trial is slow largely because eligibility is written as free text. A trial protocol lists a dozen or more inclusion and exclusion criteria; a clinician must read each one against the patient's record. Large language models (LLMs) can perform this reading, and TrialGPT [1] showed that a GPT-4-based pipeline can predict criterion-level eligibility and cite the supporting note sentences with accuracy approaching that of human experts. That work, however, relies on a proprietary model behind a paid API, which is unsuitable for privacy-sensitive records and for students without budget.

Problem statement. Given (i) a patient description whose sentences are numbered, (ii) a criterion type (inclusion or exclusion), and (iii) the criterion text, predict the expert eligibility label—including *not enough information* and *not applicable*, which are never collapsed into yes/no—and the IDs of the sentences that justify it. We require the solution to run on freely available hardware with an open-weight model, to be evaluated on a fixed, patient-grouped held-out split, and to expose the Transformer inference machinery explicitly rather than through a library switch.

Contributions.

1) A reproducible supervised adaptation of a 360M-parameter decoder (and a 135M draft) on the prescribed corpus, with before/after comparison on identical validation and test cases, plus per-class, per-criterion-type and evidence-level metrics and an error analysis.
2) A from-scratch Llama-architecture decoder with an explicit KV cache (growth, rollback, position handling), verified against the Hugging Face implementation.
3) A tiled attention routine using the online-softmax recurrence, validated against a reference attention across shapes, masks and cache offsets.
4) A greedy speculative decoder with synchronised target/draft caches whose outputs are token-identical to greedy target decoding, with acceptance statistics and an honest speed analysis.
5) A browser application that ties these parts together and lets a user compare decoding modes live.

The remainder of the paper is organised as follows. Section II reviews related work, Section III describes data, model and algorithms, Section IV the experimental setup, Section V the results, and Section VI limitations and conclusions.

II. Related Work

Clinical-trial matching. The patient descriptions in our corpus originate from the SIGIR 2016 test collection of Koopman and Zuccon [2], which pairs short synthetic patient summaries with trials judged for relevance. TrialGPT [1] decomposes matching into retrieval, criterion-level prediction with evidence, and ranking, and released the expert criterion-level annotations [3] that serve as the benchmark here (1 015 annotated patient–criterion pairs over 53 patients in the public file). We use only the criterion-level task and replace GPT-4 with a small local model; the GPT-4 outputs distributed with the dataset are discarded.

Parameter-efficient adaptation. LoRA [4] freezes the pre-trained weight W and learns a low-rank update $\Delta W = \frac{\alpha}{r} BA$, so optimizer state is kept only for the small matrices A, B. QLoRA [5] additionally stores the frozen base in 4 bits, which saves memory on GPUs but depends on CUDA kernels that are unavailable to us; at 360M parameters fp32 weights occupy only about 1.4 GB, so we use plain LoRA.

Small open decoders. SmolLM2 [6] provides 135M, 360M and 1.7B Llama-style [7] models with RMSNorm [8], SwiGLU [9], rotary embeddings [10] and grouped-query attention [11], all sharing one tokenizer—a prerequisite for speculative decoding. Qwen2.5-0.5B [12] is a comparable alternative with a much larger vocabulary and no smaller sibling.

Efficient attention. Standard attention [13] materialises a $T \times T$ score matrix. The online normaliser [14] computes softmax in one pass with a running maximum and sum; combined with tiling this yields memory-efficient exact attention [15] and its IO-aware GPU realisation FlashAttention [16]. We implement the recurrence in PyTorch for correctness and analysis, not for speed.

KV caching and speculative decoding. Autoregressive generation recomputes nothing if past keys and values are cached; managing that cache is a central serving problem [17], [18], and multi-query/grouped-query attention shrinks it [11], [19]. Speculative decoding [20], [21] lets a cheap draft model propose several tokens that the target verifies in one forward pass; with greedy decoding the output equals the target's own greedy output, and the speed-up depends on the acceptance rate and the draft/target cost ratio.

III. Materials and Methods

A. Dataset, Split and Leakage Control

We use only the TrialGPT Criterion Annotations file (1 015 rows, 53 patients) [3]. Each row holds a patient note whose sentences are numbered (`0. ...`), a trial identifier and title, the criterion type and text, the expert label, and the expert evidence sentence IDs. The file also contains GPT-4 explanations, sentence IDs, labels and a correctness flag; these four columns are dropped at load time (`LEAKY_COLUMNS` in `src/data.py`) so they cannot reach a prompt or a metric. The upstream `training` flag is ignored.

The split follows the assignment protocol: the 53 unique patient IDs are sorted, shuffled in place with `numpy.random.default_rng(42)`, and the first 37, next 8 and last 8 IDs form train, validation and test. A programmatic check asserts that the three ID sets are disjoint, jointly cover all patients and account for every annotation; the manifest is saved as `data/processed/split_manifest.json`. Table I gives the resulting sizes. One annotation (ID 883) has no criterion text in the source file, so there is nothing to predict; it stays in the manifest counts but is excluded from training and evaluation, leaving 602/206/206 usable rows. Table II shows the label distribution: the classes *excluded* and *not included* are rare, and the test split contains no *excluded* example at all, which affects how macro-F1 must be read.

TABLE I
Patient-grouped split (seed 42).

Split	Patients	Annotations	Incl.	Excl.
train	37	602	215	387
validation	8	206	66	140
test	8	206	75	131

TABLE II
Expert label distribution per split (usable rows).

Label	Train	Val.	Test
included	88	35	27
not included	13	3	7
excluded	8	2	0
not excluded	291	101	91
not enough info.	174	53	66
not applicable	28	12	15

B. Task Formulation, Prompt and Output Schema

The input is rendered through the model's ChatML template with a fixed system message, the numbered note, the criterion type, the criterion text and the list of labels allowed for that type (four per type: *included/not included* or *excluded/not excluded*, plus *not enough information* and *not applicable*). The target is a compact JSON object,

```
{"label": "not excluded", "evidence": [3, 5]}
```

followed by the end-of-turn token. Evidence IDs are the original sentence IDs of the note, sorted ascending; an empty list is a legitimate answer. The parser accepts the JSON form, falls back to a lenient search for a label string, and marks anything else *invalid*, which always counts as an error. Notes are short (mean prompt+answer length 301 tokens, maximum 580 with the SmolLM2 tokenizer), so with a 1 024-token limit **no example is truncated** (0 truncations); the code nevertheless counts truncation events and would report them.

C. Model Candidates and Selection

We study two hardware tiers with the same model family. Table III lists the candidates with properties computed directly from each model's published `config.json`: parameter count, fp32 weight memory (parameters $\times$ 4 bytes), layers, query/key-value heads, vocabulary, context window, the fp32 KV-cache

TABLE III
CANDIDATE DECODERS, COMPUTED FROM THEIR CONFIGURATION FILES (FP32 WEIGHTS IN DECIMAL GB; KV = CACHE BYTES PER TOKEN IN FP32).

Model	Params	fp32 (GB)	Layers	Heads/KV	Vocab	Ctx	KV/token (kB)	Role
SmolLM2-1.7B	1.71 B	6.85	24	32/32	49 152	8 192	393	target (A)
SmolLM2-360M	362 M	1.45	32	15/5	49 152	8 192	82	target (B) / draft (A)
SmolLM2-135M	135 M	0.54	30	9/3	49 152	8 192	46	draft (B)
Qwen2.5-0.5B	494 M	1.98	24	14/2	151 936	32 768	25	rejected

cost per token ($2 \cdot L \cdot H_{kv} \cdot d_{head} \cdot 4$ bytes), and the role each model plays in this work. All four are Apache-2.0 licensed decoders (Llama-style for SmolLM2, Qwen2-style for Qwen2.5-0.5B). The figures are derived, not benchmarked; the training and inference times that actually decided the configuration are reported in Section V.

Configuration A (GPU, headline). Target **SmolLM2-1.7B-Instruct**: 1.71 B parameters, 24 layers, hidden size 2 048, 32 query heads with 32 key/value heads (multi-head attention, 64-dimensional heads), SwiGLU feed-forward of width 8 192, rotary embeddings ($\theta = 1.3 \times 10^5$), 8 192-token context, tied embeddings, 49 152-token vocabulary; about 6.85 GB in fp32. It trains on a free 15 GB T4 with LoRA and fp16 autocast. Draft **SmolLM2-360M-Instruct** (same tokenizer). Larger options (e.g. 7–8B models) would require 4-bit quantisation and CUDA-only libraries and were not considered.

Configuration B (CPU). Target **SmolLM2-360M-Instruct**: 32 layers, hidden size 960, 15 query heads sharing 5 key/value heads (grouped-query attention), feed-forward width 2 560, 8 192-token context; about 1.45 GB in fp32 and trainable in hours on four CPU cores. Draft **SmolLM2-135M-Instruct** (30 layers, hidden size 576, 9 heads / 3 KV heads), same 49 152-token vocabulary. Qwen2.5-0.5B [12] (24 layers, hidden size 896, 14 heads / 2 KV heads, 151 936-token vocabulary, 32 768 context) was the alternative decoder: it has a three times larger embedding/softmax, query/key/value biases that our decoder would need to support, and no smaller member of its family to act as a same-tokenizer draft, so it was rejected (Table III).

D. Fine-Tuning

LoRA adapters (rank $r = 16$, $\alpha = 32$, dropout 0.05) are attached to all seven linear projections of every layer (`q,k,v,o,gate,up,down`): 18.09M trainable parameters for the 1.7B target (1.05%) and 8.68M for the 360M model (2.3%). The objective is token-level cross-entropy computed *only on the answer tokens* (the JSON string and end-of-turn token); prompt tokens are masked with -100, and the loss is normalised by the number of answer tokens in each optimizer batch. We use AdamW (weight decay 0.01), 10% linear warm-up followed by cosine decay, gradient clipping at 1.0, effective batch size 16 via gradient accumulation, and seed 42. Configuration A: learning rate 2×10^{-4}, 3 epochs for the 1.7B target (micro-batch 1), 3×10^{-4} and 4 epochs for the 360M draft (micro-batch 4), fp16 autocast with a gradient scaler on a T4. Configuration B: learning rate 3×10^{-4}, 2 epochs for the 360M target and 3 for the 135M draft, fp32 on CPU. After each epoch the script records validation loss and runs greedy generation on the validation split to obtain accuracy, macro-F1 and evidence F1; the adapter with the highest validation macro-F1 (ties broken by lower validation loss) is kept. The test split is never read by the training script. At inference the adapter is folded into the base weights as $W \leftarrow W + \frac{\alpha}{r} BA$ by our own loader (verified identical to the PEFT merge: maximum logit difference 0 on a test prompt), so no PEFT dependency is needed at serving time; the adapters are 72 MB (1.7B), 35 MB (360M) and about 13 MB (135M).

E. Explicit Decoder and KV Cache

To control caching, masking and the attention kernel, we re-implemented the Llama forward pass (`src/minillama.py`) reading the Hugging Face safetensors. Correctness is checked against `LlamaForCausalLM` (Section V-F). Fig. 1(a) shows one decoder step: new tokens are embedded, normalised and projected to queries, keys and values; rotary embeddings are applied with absolute positions; the rotated keys and values are written into the per-layer cache; the selected attention back-end reads the whole valid cache under the causal rule; and the usual output projection, residuals and SwiGLU feed-forward follow. Fig. 1(b) summarises the speculative round described in Section III-G.

Positions. Rotary embeddings rotate each query and key by its absolute position. For a step that feeds T new tokens after L cached ones, positions are $L, \ldots, L+T-1$. Keys are rotated *before* they are written to the cache, so cached keys already carry their position and are never rotated again; queries are rotated the same way and never cached.

Masking. Query i of the new block has absolute position $L+i$ and may attend to cache entry j iff $j \le L+i$. During prefill ($L = 0$) this is the usual lower-triangular mask; during one-token decoding ($T = 1$) it admits the entire cache. The same `q_offset` argument drives the mask in all attention back-ends.

Growth and rollback. Each layer owns a pre-allocated key tensor and value tensor of shape (B, H_{kv}, C, d) with a shared length counter. New keys/values are written at slots $[L, L+T)$; if $L+T > C$ the capacity doubles (amortised $O(1)$ copies per token, in contrast to one `torch.cat` per step). Rolling back n tokens is just setting the counter to $L-n$, which later writes overwrite—this is exactly what speculative decoding needs. With 32 layers, 5 KV heads and $d = 64$, one cached token costs $2 \cdot 32 \cdot 5 \cdot 64 \cdot 4 = 82{,}000$ bytes in fp32 (about 80 KB), i.e. $3\times$ less than with 15 KV heads.

Numerical differences. Cached and uncached paths compute the same mathematical function but with different matrix shapes (e.g. a $1 \times d$ query against L keys versus a full $T \times T$ product), which changes floating-point summation order. Logits can therefore differ at the 10^{-5} level, and a greedy argmax can flip only when the top-two logits are closer than that. We test token equality empirically (Section V-F).

F. Reference and Tiled Online-Softmax Attention

The reference routine computes $S = QK^\top/\sqrt{d}$, applies the causal mask $M_{ij} = [j \le i + L]$ by setting masked scores to $-\infty$, subtracts the row maximum, exponentiates, normalises

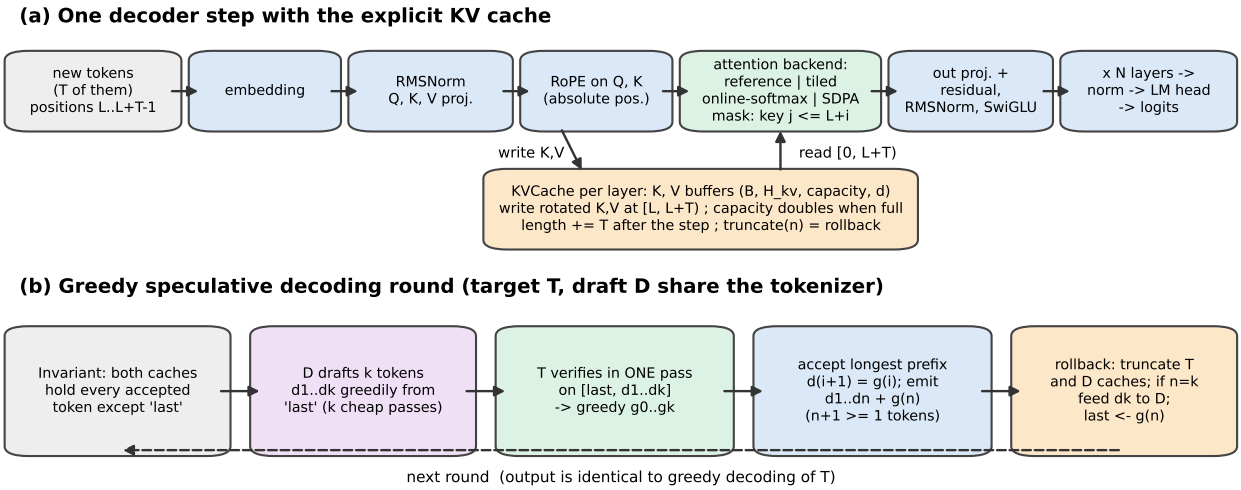


Fig. 1. Architecture of our decoder implementation. (a) One forward step with the explicit, growable KV cache and the pluggable attention back-end (reference, tiled online-softmax, or PyTorch SDPA for comparison). (b) One round of greedy speculative decoding with cache rollback; the output equals greedy decoding of the target.

Algorithm 1 Tiled attention with online softmax (one head)

for each query block Q_b (rows r) **do**
  $m \leftarrow -\infty,\ \ell \leftarrow 0,\ o \leftarrow 0$
  for each key block K_c, V_c with c not strictly above the diagonal **do**
    $S \leftarrow Q_b K_c^\top / \sqrt{d}$; mask entries with key position $>$ query position
    $m' \leftarrow \max(m, \mathrm{rowmax}(S));\quad \alpha \leftarrow e^{m-m'};\quad P \leftarrow e^{S-m'}$
    $\ell \leftarrow \alpha\ell + \mathrm{rowsum}(P);\quad o \leftarrow \alpha o + PV_c;\quad m \leftarrow m'$
  end for
  output rows $\leftarrow o/\ell$
end for

and multiplies by V; fully masked rows return zeros instead of NaN.

The tiled routine (Algorithm 1) never forms S. For each block of queries it sweeps over blocks of keys while keeping per-row statistics: the running maximum m, the running normaliser $\ell = \sum_j e^{s_j - m}$ and the running un-normalised output $o = \sum_j e^{s_j - m} v_j$. When a new key block with block maximum $\tilde{m}$ arrives, $m' = \max(m, \tilde{m})$, the old statistics are rescaled by $\alpha = e^{m-m'}$, and the new block's contributions are added with the new maximum. The final output is o/ℓ. Key blocks entirely above the causal diagonal are skipped, and grouped KV heads are broadcast to query heads block by block. PyTorch's fused `scaled_dot_product_attention` is used only as a third comparison point.

G. Greedy Speculative Decoding

Target (360M) and draft (135M) share the tokenizer, so token IDs are directly comparable. Both are held to one invariant at the start of each round: *both caches contain every accepted token except the most recent one* ($last$). A round (Algorithm 2) (1) lets the draft decode k tokens from $last$, (2) runs *one* target forward pass over $[last, d_1, \ldots, d_k]$, whose output at position i is the target's greedy token g_i after the i-th input, (3) accepts the longest prefix with $d_{i+1} = g_i$ and appends the target's own token g_n (a correction at the first mismatch, or a free bonus token if all k matched), and (4) restores the invariant: the target cache is truncated to keep only $last, d_1..d_n$; the draft cache is truncated likewise, or, if all drafts were accepted, receives d_k (which it had generated but never consumed). Each round therefore emits $n + 1 \geq 1$ tokens and, by construction, the output equals greedy decoding of the target.

Algorithm 2 One round of greedy speculative decoding

$d_1..d_k \leftarrow$ draft greedy tokens starting from $last$ (draft cache grows by k)
$g_0..g_k \leftarrow \arg\max$ of target logits on $[last, d_1..d_k]$ (target cache grows by $k{+}1$)
$n \leftarrow$ largest $n \leq k$ with $d_i = g_{i-1}\ \forall i \leq n$
emit $d_1..d_n, g_n$; target cache $\leftarrow$ keep $last, d_1..d_n$
if $n = k$ **then** feed d_k to the draft
else truncate draft cache to $last, d_1..d_n$
end if
$last \leftarrow g_n$

H. Criterion-Level Pipeline and Aggregation

Fig. 2 gives the end-to-end system: the offline training path (dataset, fixed split, prompt construction, LoRA fine-tuning, adapter selection) and the inference path used by the evaluation scripts and the browser application (input validation and sentence numbering, our decoder with the merged adapter under a chosen decoding mode, parsing, evidence extraction and explanation, rule-based aggregation, comparison view and report export).

For every criterion the pipeline returns: the criterion, predicted label, cited sentence IDs (IDs absent from the note are discarded and reported), the extracted text of those sentences,

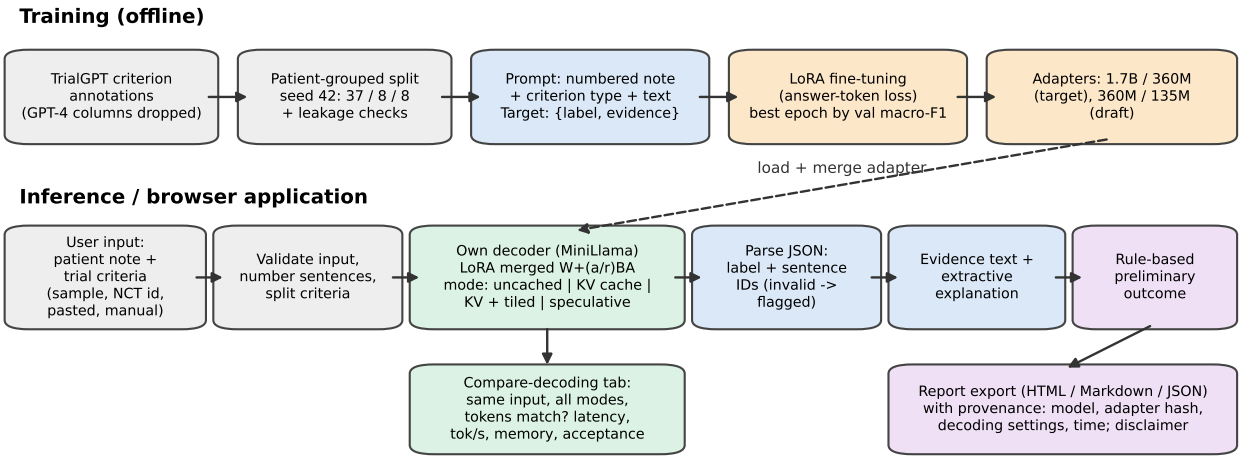


Fig. 2. System architecture and application workflow. Top: offline training with the prescribed dataset and fixed split. Bottom: inference as used by evaluation and by the browser application.

and an explanation. The explanation is a deterministic template that restates the label and quotes only the cited sentences; for *not enough information* it states that missing mention is not evidence either way. We deliberately do not train on or generate free-text rationales, because the only rationales in the corpus are GPT-4 outputs that the protocol forbids. A patient–trial *preliminary outcome* is an explicit rule over the criterion labels, not a learned or ground-truth label: any inclusion *not included* or exclusion *excluded* gives "likely not eligible"; otherwise any *not enough information* or invalid output gives "needs more information"; otherwise "potentially eligible".

I. Application

The application (FastAPI back-end, single-page browser UI, `app/`) loads the target and draft once, then lets a user paste a note (auto-numbered when needed), choose a held-out sample trial, fetch one from ClinicalTrials.gov by NCT ID (metadata only), or paste an eligibility block that is split into criteria. It shows each criterion's label, cited sentences highlighted in the numbered note, the explanation and raw output, and a rule-based overall outcome; reports are exported as HTML, Markdown or JSON with model, adapter hash, decoding settings and time stamp. A *Compare decoding* tab runs the current input through all four decoding modes on the user's machine and tabulates tokens-match, latency, throughput, memory and acceptance rate; an *Experiments* tab renders the stored result files. Invalid input (short notes, unknown criterion types, empty reports, unreachable trial registry) is rejected with explanatory messages. A banner states that this is an educational screening prototype and not a clinical enrollment decision.

IV. Experimental Setup and Metrics

A. Compute Environment and Memory Limits

Configuration A (GPU) ran on a free Kaggle notebook with one NVIDIA Tesla T4 (15 GB; the notebook exposes two, only one is used), PyTorch 2.11 (CUDA 13 build), a recent Hugging Face Transformers 5.x [22] and PEFT 0.21 for training and for reference checks. Training used fp16 autocast with fp32 master weights and LoRA parameters; *all inference, evaluation and benchmarks run in fp32* so that exact-equality tests are meaningful. The 1.7B target needs about 6.85 GB for fp32 weights, which together with activations left too little room for a micro-batch of 4 on the first attempt (an out-of-memory error), hence micro-batch 1 with 16 accumulation steps.

Configuration B (CPU) ran on a laptop: Intel Core i5-1135G7 (4 cores / 8 threads), 16 GB RAM, Windows 11, no GPU, PyTorch 2.14 (CPU build), Transformers 5.22 and PEFT 0.21. Everything uses fp32 on four threads, because CPU bf16 matrix multiplication is slower than fp32 on this processor and would add rounding noise to the exact-equality tests. The 360M target needs roughly 1.45 GB for weights and, during LoRA training with activations and AdamW state, a peak working set of about 4 GB; the 135M draft needs about 0.55 GB. Training time on this machine is the practical limit that fixed the model size (Section III-C).

Both configurations use exactly the same data split, prompts, parser, metrics and code; only the model pair, device and training schedule differ. The CPU configuration also serves as the model of the locally hosted application.

B. Classification Metrics

We report accuracy, macro-F1 and per-class precision, recall and F1 with support, plus confusion matrices, for all test criteria and separately for inclusion and exclusion criteria. Macro-F1 averages over the classes that occur either in the gold labels or in the predictions of the scope under consideration; a class absent from both is skipped, so the missing *excluded* class in the test split does not silently count as a zero. An *invalid* model output is counted as wrong and is never mapped onto a label. Because the test split contains only 8 patients whose

rows are correlated, we also give 95% patient-level bootstrap intervals (2 000 resamples of patients).

C. Evidence-Sentence Metrics

Evidence is scored on sentence IDs as set overlap. *Micro* scores pool true/false positives and false negatives over all records. *Macro* scores average per-record precision, recall and F1 in two ways: (a) over records with at least one gold sentence ("evidence-bearing"), where predicting nothing yields precision 0 by convention, and (b) over all records, where an empty gold set matched by an empty prediction counts as a perfect record. Records with empty gold evidence are additionally reported on their own: the fraction for which the model also cited nothing and the number of spurious citations it made. About 145 of the 61+145 test records have no annotated evidence, so the policy matters and we show both views. Absence of cited evidence is never converted into an eligibility judgement.

D. Decoding Settings

Every accuracy number uses deterministic greedy decoding (argmax), at most 48 new tokens, stopping at the end-of-turn token, with the same prompt for the base and the adapted model. The base model is evaluated zero-shot with the identical prompt and parser on exactly the same validation and test rows; it receives no few-shot examples and no adapter. Hyper-parameters were chosen once from the configuration in Section III-D and not tuned on the test split.

E. Inference Benchmarks

KV cache. For prompt lengths $\{64, 256, 512\}$ tokens (real prompts built from test criteria, concatenated and cut) and output lengths $\{16, 64, 128\}$, the uncached reference, the cached path and the cached path with the tiled kernel generate with EOS suppressed so that all paths emit exactly the same number of tokens. These benchmarks were run on the GPU of Configuration A (1.7B target); the laptop CPU was used for training, evaluation and the live comparison inside the application (Section V-J), not for stored benchmark tables. Each benchmark point runs in a fresh process with one warm-up generation and three timed repetitions; we report medians of prefill time (time to the first token), decode time, tokens per second and peak memory growth. On the GPU the memory figure is the exact CUDA-allocator peak minus the allocation before the call (`torch.cuda.max_memory_allocated`); in the application on a CPU it is the resident-set size sampled every 2 ms while generating (peak minus the value before the call), a proxy because the CPU allocator has no peak counter. Greedy token sequences must be identical across paths.

Attention. Shapes follow the 360M model (15 query heads, 5 KV heads, $d = 64$) for $T \in \{128, \ldots, 4096\}$ on the GPU, in prefill mode ($T_q = T_k = T$) and decode mode ($T_q = 1$ against T cached keys), with five repetitions per point and the same fresh-process memory probe. Errors are the maximum and mean absolute difference to the reference output.

Speculative decoding. Thirty-two test-criterion prompts (sampled with a fixed seed; only speed and token equality are measured, no model selection uses them) are decoded by cached greedy target decoding and by our speculative decoder for $k \in \{2, 4, 6, 8\}$, once with the LoRA-adapted draft and once with the un-adapted draft. We report token equality, acceptance rate (accepted drafted tokens / drafted tokens), mean accepted tokens per round, target passes per emitted token, mean latency, tokens per second and speed-up. Two output regimes are used: the *natural* task output (about 15 tokens of JSON) and a *forced* 64-token output (EOS suppressed) that exposes behaviour on longer generations. Hugging Face assisted generation serves as an independent equality check on a subset.

TABLE IV
CONFIG. A TARGET (SMOLLM2-1.7B + LORA): VALIDATION AFTER EACH EPOCH (* = SELECTED).

Epoch	Val. loss	Val. acc.	Val. macro-F1	Val. evid. F1
1	0.190	0.675	0.309	0.306
2*	0.155	0.714	0.365	0.374
3	0.156	0.694	0.326	0.391

V. RESULTS AND DISCUSSION

We report two configurations that use the identical split, prompt, parser and metrics. **Configuration A** (GPU, Kaggle T4) fine-tunes SmolLM2-1.7B as target with SmolLM2-360M as draft and carries the headline accuracy results; **Configuration B** (CPU, laptop) uses the 360M target with the 135M draft and is the model that the locally hosted application loads by default (Section V-I). All numbers below come from files written by the scripts in the repository.

A. Fine-Tuning and Model Selection (Config. A)

Table IV lists validation behaviour after each epoch and Fig. 3 the training curves. Answer-token training loss falls from about 0.40 to about 0.12 for the 1.7B target, and validation loss falls from 0.190 to 0.155 between epochs 1 and 2 and then stays flat (0.156). Validation macro-F1 peaks at epoch 2 (0.365) and drops at epoch 3 (0.326), and validation accuracy follows the same pattern (0.714 vs. 0.694), so the epoch-2 adapter was kept for the target; the same criterion selected epoch 2 for the 360M draft (Table V). Because macro-F1 over six classes on 206 validation rows is noisy (the curves in Fig. 3 move by several points between epochs while the loss is flat), the selection is made on a weak signal; we report it as observed rather than as a tuned optimum. Training took 0.3 h (target) and 0.1 h (draft) on one T4 with fp16 autocast, micro-batch 1 and 16 accumulation steps for the 1.7B model (to fit 15 GB), and micro-batch 4 with 4 accumulation steps for the draft; the effective batch is 16 in both cases.

B. Before and After Fine-Tuning (Config. A)

Table VI compares, on *identical* validation and test rows with the same prompt, greedy decoding and parser, a trivial majority-label baseline, the base model used zero-shot, and the fine-tuned model; Fig. 4 visualises the test columns. The base model is almost never right: test accuracy 0.146 (95%

TABLE V
CONFIG. A DRAFT (SMOLLM2-360M + LORA): VALIDATION AFTER EACH EPOCH.

Epoch	Val. loss	Val. acc.	Val. macro-F1	Val. evid. F1
1	0.220	0.621	0.215	0.124
2*	0.192	0.718	0.341	0.325
3	0.186	0.641	0.259	0.063
4	0.184	0.689	0.323	0.262

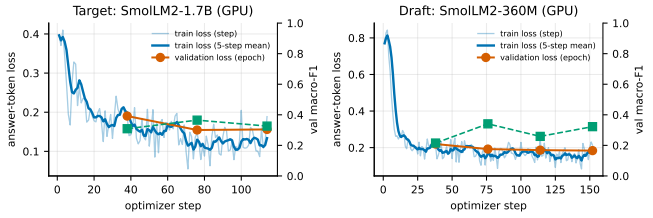


Fig. 3. Config. A training curves: step and smoothed training loss, epoch-level validation loss (red) and validation macro-F1 (green, right axis).

patient-bootstrap interval 0.104–0.198) and macro-F1 0.111. Its outputs are well-formed JSON (only 1 invalid output), so the failure is semantic: it commits to a decisive label, and on all 131 exclusion rows it never produces the correct one (exclusion accuracy 0.000). Fine-tuning lifts test accuracy to 0.709 (interval 0.578–0.818) and macro-F1 to 0.460 (interval 0.359–0.554), with no invalid outputs.

Two cautions are needed to read this correctly. First, a trivial predictor already attains 0.626 accuracy on the test split because *not excluded* and *not enough information* dominate; the accuracy interval of the fine-tuned model (lower end 0.578) therefore overlaps the majority baseline, so we cannot claim a statistically supported accuracy gain over guessing the common label. The macro-F1 gain is clearer (0.460 vs. 0.272, interval lower end 0.359), because macro-F1 rewards the minority classes. Second, the gain is entirely on inclusion criteria: inclusion accuracy rises from the majority baseline's 0.507 to 0.747 and macro-F1 from 0.168 to 0.516, whereas exclusion accuracy (0.687) is *below* the majority baseline's 0.695 and macro-F1 is unchanged (0.271 vs. 0.273). The reason is visible in the confusion matrices below: for exclusion criteria the model predicts *not excluded* for 130 of 131 rows.

C. Per-Class Results and Inclusion vs. Exclusion (Config. A)

Table VII and Fig. 5 give per-class precision, recall and F1 and the confusion matrices overall and by criterion type (rows are expert labels, columns predictions). *Included* is handled well (P 0.79, R 0.70, F1 0.75 on 27 rows) and *not excluded* has high recall (0.99) but only 0.69 precision because 28 *not enough information* and 12 *not applicable* exclusion rows are also labelled *not excluded*. *Not enough information* reaches F1 0.62 (inclusion: 36 of 38 correct), *not applicable* has precision 1.0 but recall 0.07 (1 of 15), and *not included* (7 rows) is never predicted. The *excluded* class has no test support, so it contributes neither to recall nor to macro-F1 (Section IV-B). The base model's per-class table (Table VIII) shows the contrast: it spreads predictions over decisive labels with near-zero precision. Table VII should therefore be read as: reliable on *included* and *not enough information* for inclusion criteria, essentially majority-class behaviour for exclusion criteria, and no ability on the rarest classes.

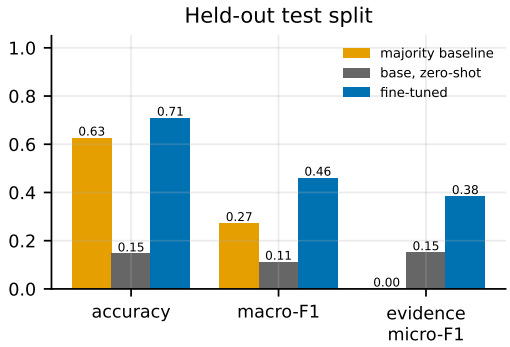


Fig. 4. Config. A test results: majority baseline, base zero-shot and fine-tuned model.

D. Evidence-Sentence Localisation (Config. A)

Table IX reports sentence-ID precision, recall and F1. 145 of the 206 test records have *no* annotated evidence sentence and only 61 have at least one, which dominates every aggregate. With micro pooling the fine-tuned model reaches P 0.538, R 0.298, F1 0.384 (interval 0.276–0.476) against the base model's 0.151: the base model cites many sentences (recall 0.447, precision 0.091), the adapted model cites few but more correct ones. On evidence-bearing records the macro F1 is 0.277; if empty-gold records count as correct when nothing is cited, macro F1 rises to 0.757 purely because the model leaves 95.9% of the empty-gold records empty (10 spurious citations in total), which shows why that second number must not be read as localisation skill. The split by type is telling: inclusion micro-F1 is 0.491 while exclusion micro-F1 is 0.000, because the model never cites a sentence for an exclusion criterion. Cited IDs are always valid sentence IDs of the note, and the application only displays text that exists in the note.

E. Error Analysis (Config. A)

Of the 206 test criteria, 60 (29%) receive a wrong label and a further 35 have a correct label with a different evidence set; 111 are correct in both respects. Table X categorises every non-correct case and Table XI lists twelve representative, distinct cases from different patients, trials and categories (full text in `results/error_analysis_gpu_finetuned_test.md`).

Missing information treated as "not excluded". The largest category is false decision: 42 rows whose expert label is *not enough information* (30) or *not applicable* (12) but where the model commits to a decisive label; 40 of the 42 are exclusion criteria answered *not excluded*. E1 ("Pregnancy", expert: not applicable) and E3 ("BMI > 40", expert: not enough information) are typical: the note contains nothing about the

TABLE VI
CONFIG. A: MAJORITY BASELINE, BASE ZERO-SHOT AND FINE-TUNED SMOLLM2-1.7B ON THE SAME ROWS (ACC. = ACCURACY, MF1 = MACRO-F1; INVALID = UNPARSEABLE OUTPUTS).

Split	Model	Acc.	mF1	Incl. acc.	Incl. mF1	Excl. acc.	Excl. mF1	Invalid
Val.	majority label	0.621	0.215	0.409	0.145	0.721	0.210	0
	base zero-shot	0.170	0.131	0.500	0.206	0.014	0.007	0
	fine-tuned	0.714	0.365	0.667	0.344	0.736	0.269	0
Test	majority label	0.626	0.272	0.507	0.168	0.695	0.273	0
	base zero-shot	0.146	0.111	0.400	0.182	0.000	0.000	1
	fine-tuned	0.709	0.460	0.747	0.516	0.687	0.271	0

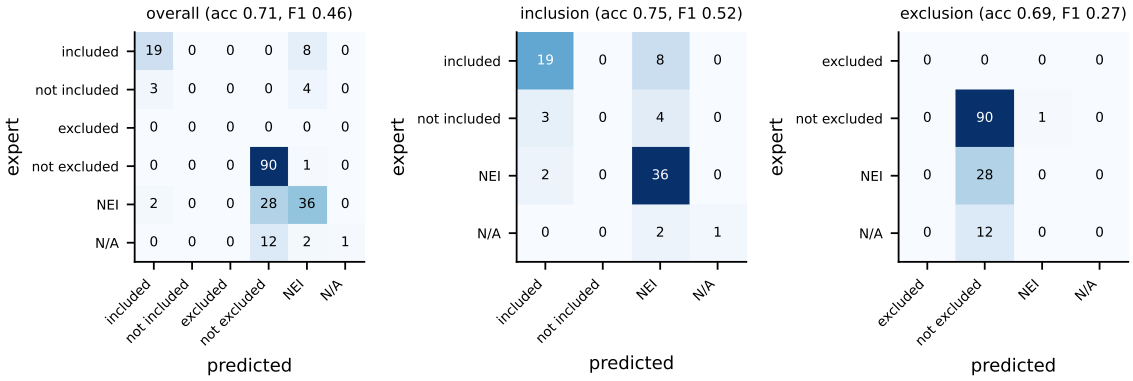


Fig. 5. Config. A confusion matrices on the test split (NEI = not enough information, N/A = not applicable).

TABLE VII
CONFIG. A FINE-TUNED MODEL, TEST SPLIT: PER-CLASS PRECISION (P), RECALL (R), F1 AND SUPPORT.

Scope	Class	P	R	F1	Support
inclusion	included	0.792	0.704	0.745	27
	not included	0.000	0.000	0.000	7
	not enough info.	0.720	0.947	0.818	38
	not applicable	1.000	0.333	0.500	3
exclusion	excluded	0.000	0.000	0.000	0
	not excluded	0.692	0.989	0.814	91
	not enough info.	0.000	0.000	0.000	28
	not applicable	0.000	0.000	0.000	12

TABLE VIII
CONFIG. A BASE MODEL (ZERO-SHOT), TEST SPLIT.

Scope	Class	P	R	F1	Support
inclusion	included	0.409	1.000	0.581	27
	not included	0.000	0.000	0.000	7
	not enough info.	1.000	0.079	0.146	38
	not applicable	0.000	0.000	0.000	3
exclusion	excluded	0.000	0.000	0.000	0
	not excluded	0.000	0.000	0.000	91
	not enough info.	0.000	0.000	0.000	28
	not applicable	0.000	0.000	0.000	12

topic and the model, instead of reporting the gap, defaults to the label that is correct most often. This is exactly the behaviour

TABLE IX
CONFIG. A EVIDENCE-SENTENCE METRICS ON THE TEST SPLIT (MICRO, AND MACRO OVER EVIDENCE-BEARING OR ALL RECORDS).

Model	Scope	P	R	F1	Type
base	overall	0.091	0.447	0.151	micro
base	inclusion	0.158	0.500	0.240	micro
base	exclusion	0.041	0.344	0.074	micro
base	overall	0.267	0.412	0.295	macro (gold$\neq \emptyset$)
base	overall	0.079	0.122	0.087	macro (all records)
fine-tuned	overall	0.538	0.298	0.384	micro
fine-tuned	inclusion	0.538	0.452	0.491	micro
fine-tuned	exclusion	0.000	0.000	0.000	micro
fine-tuned	overall	0.290	0.295	0.277	macro (gold$\neq \emptyset$)
fine-tuned	overall	0.761	0.762	0.757	macro (all records)

the task warns against—absence of evidence being read as eligibility—so *not excluded* outputs without cited evidence should be shown to users as unverified, which the application does by displaying “no supporting sentence cited”.

Uncited decisive predictions. 130 test predictions carry a decisive label with no cited sentence. For 114 of them the expert also cites nothing, which is consistent with the dataset (about three quarters of *not excluded* labels have empty evidence), but 16 have expert evidence that the model failed to cite (E4, E8): these are the contradictory cases where a label is given without the supporting sentence that exists in the note.

TABLE X
CONFIG. A TEST ERRORS BY CATEGORY (ALL 206 ROWS; "CORRECT" MEANS LABEL AND EVIDENCE BOTH MATCH).

Error category	Count
correct	111
false decision	42
evidence only	35
missed decision	13
polarity flip	3
na confusion	2

Missed decisions. 13 rows have a decisive expert label but the model answers *not enough information* (E7, E8): for example "Shock or ARDS" is marked *included* by the expert based on hypotension and tachycardia (sentences 2–3), while the model says the note lacks information. These failures need clinical inference from vital signs, which a small model trained on 602 rows does not acquire.

Polarity flips and not applicable *confusions* are rare (3 and 2), and the *not included* class (7 rows) is mostly answered *included* (3) or *not enough information* (4). *Evidence-only* errors (35) divide into over-citation, where a *not enough information* answer cites the first five sentences although the expert cites none (E5), and under-citation, where the single relevant sentence is missed (E6).

F. KV-Cached Generation (Config. A)

Table XII compares the uncached reference, the explicit KV cache with PyTorch SDPA attention, and the cache combined with our tiled kernel for prompts of 64, 256 and 512 tokens and 16, 64 and 128 generated tokens on the T4, in fp32 on the 1.7B model. **Correctness:** the greedy token sequences of all three paths are identical in 27/27 configurations, including 512+128 tokens, even though the cached path multiplies a single query against the cache while the reference recomputes full matrices; no mismatch needed investigation, and the unit tests on smaller models additionally match the Hugging Face logits to 6×10^{-5}. **Speed:** prefill time is the same for all paths (0.09–0.7 s, growing with prompt length) as expected, because prefill computes the same thing. Decode is where the cache matters: the uncached path re-reads and recomputes the whole prefix at every step, so its throughput (prefill included) is only 4.3–6.6 tokens/s for 64-token prompts and falls to 1.2–1.3 tokens/s for 512-token prompts, whereas the cached path stays between 14 and 30 tokens/s, i.e. 18.7× faster end to end at 512+128 tokens (Fig. 6). The cached tokens/s itself declines mildly with prompt length (30.1 at 64 tokens to 24.8 at 512) because every step attends to more cached keys. **Memory:** the cache is the price paid for this speed. The 1.7B model uses multi-head attention with 32 KV heads, so one cached token costs $2 \cdot 24 \cdot 32 \cdot 64 \cdot 4 = 393$ kB in fp32 (the measured 239.6 MB for 639 tokens agrees), and the measured peak allocation growth is 289–331 MB with the cache versus 91–110 MB without at 512 tokens (Fig. 7); this includes the pre-allocated buffer of 256 or more slots. For comparison, the grouped-query 360M model needs only 82 kB per token. **Tiled attention inside the decoder:** the cached path with our tiled kernel produces identical tokens but is 1.6–4.5× slower than SDPA (5.5 vs. 24.8 tokens/s at 512+128) because every decode step runs a Python loop over key blocks, an overhead discussed next.

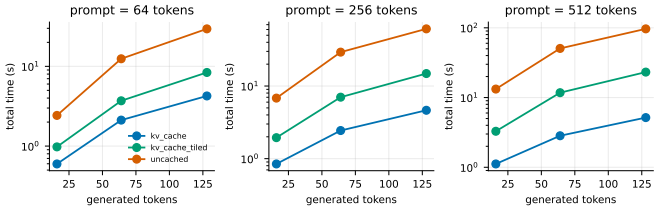


Fig. 6. Config. A: total generation time (log scale) against generated tokens for three prompt lengths.

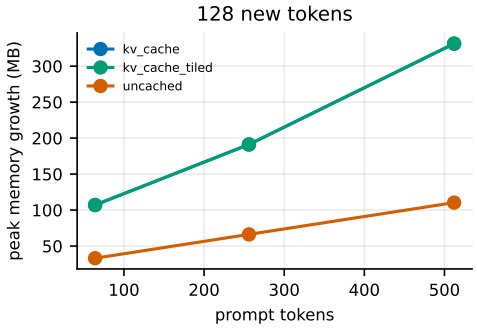


Fig. 7. Config. A: peak allocation growth against prompt length at 128 new tokens.

G. Tiled Online-Softmax Attention (Config. A)

Table XIII and Fig. 8 validate and benchmark the kernels at the head shape of the 360M model (15 query heads, 5 KV heads, $d = 64$) with random inputs. **Numerical agreement:** against the float32 reference the tiled kernel's maximum absolute error never exceeds 9.6×10^{-7} across $T = 128 \ldots 4096$ in prefill and $T = 128 \ldots 2048$ in decode (worst error over all measured points, including SDPA: 1.4e-06), with a mean error around 2×10^{-8}; unit tests add a hand-computed example, grouped heads, non-zero cache offsets, left padding and extreme score magnitudes where naive softmax overflows. **Memory:** the reference path's peak allocation grows quadratically (196 MB at T=1024, 754 MB at 2048, 2 956 MB at 4096) whereas the tiled kernel's grows roughly linearly (11, 22, 45 MB), which is the intended effect of never forming the $T \times T$ score matrix. **Speed:** the tiled kernel is *slower*: at T=1024 it needs 98 ms against 5.9 ms for the reference and 2.8 ms for the fused SDPA kernel, and 1.6 s against 83 ms at 4096. This is expected for an educational PyTorch implementation: each tile launches half a dozen small kernels and Python-level loop iterations, reads and writes the running statistics from global memory, and cannot overlap data movement with compute as a fused FlashAttention-style kernel does with on-chip SRAM [16]. The tile-size sweep supports this explanation (Fig. 9): at T=1024

TABLE XI
CONFIG. A: TWELVE REPRESENTATIVE ERRORS (EVIDENCE LISTS ARE SENTENCE IDS; EXPERT/MODEL COLUMNS SHOW LABEL AND IDS).

ID	Type	Criterion (abridged)	Expert	Model
E1	false decision	Pregnancy	not applicable []	not excluded []
E2	false decision	Patients were moribund.	not enough info. [0, 1, 2, 3, 4]	not excluded []
E3	false decision	BMI¿ 40 kg/m2.	not enough info. []	not excluded []
E4	evidence only	Had been performed central venous puncture or catheterization before.	not excluded [0]	not excluded []
E5	evidence only	bilateral pulmonary infiltrates or a chest radiograph consistent with edema;	not enough info. []	not enough info. [0, 1, 2, 3, 4]
E6	evidence only	Patients with liver space occupying lesion (LOE) that require extended hepatic resection.	not enough info. [3]	not enough info. []
E7	missed decision	Patients undergoing elective carotid endarterectomy	not included []	not enough info. []
E8	missed decision	Patients were included if they were diagnosed with Shock, Acute respiratory distress syndr…	included [2, 3]	not enough info. []
E9	missed decision	Present with active SLE disease with at least one of the following symptoms: i) Arthritis …	included [0, 1]	not enough info. []
E10	polarity flip	Patients undergoing open abdominal aortic aneurysm repair	not included []	included [0]
E11	na confusion	If female, using birth control	not applicable [3]	not enough info. []
E12	na confusion	female patients have adequate birth-control method	not applicable []	not enough info. []

the time drops from 1 560 ms with 16×16 tiles to 98 ms at 64, 30 ms at 128 and 7.6 ms at 256, nearly matching the reference, while the error is unchanged ($\leq 9.5 \times 10^{-7}$)—the number of Python-level iterations, not arithmetic, dominates. In decode mode (T_q=1) there is no score matrix to save, so the reference (0.65 ms) is even faster than SDPA (1.04 ms) at T=2048, since both are launch-bound.

H. Speculative Decoding (Config. A)

Tables XIV and XV and Fig. 10 (acceptance and speed-up against the draft length k) compare cached greedy decoding of the 1.7B target against our speculative decoder with the fine-tuned 360M draft and with the un-tuned 360M draft, on 32 test prompts (speed only; no model selection used these prompts). **Correctness:** outputs are token-identical to greedy target decoding for *every* prompt and every setting (32/32 in all 16 rows), and an independent check with Hugging Face assisted generation on six prompts agrees: 6/6 prompts: HF assisted == ours == cached greedy. **Acceptance:** with natural task output (about 13 tokens of JSON) the fine-tuned draft is accepted 92–98% of the time, against 40–64% for the un-tuned draft, so adapting the draft on the same task raises acceptance by 30–50 points; with forced 64-token outputs, where the continuation after the JSON is unconstrained, acceptance drops to 33–65% for both drafts and the benefit of adaptation disappears. Acceptance falls with the draft length k in all cases. **Latency:** despite high acceptance, speculative decoding is *slower* than plain greedy decoding in every configuration: the best speed-up is 0.81× (k=6, finetuned draft) for natural output and as low as 0.39× for forced long output. The cause is the cost ratio. On a GPU single-token decoding is limited by per-layer kernel launches and memory latency, not by arithmetic, so latency is roughly proportional to depth; the 360M draft has *32* layers while the 1.7B target has only *24*, so a draft step costs about as much as a target step. The speed-up model of Leviathan et al. [20], $(1-\alpha^{k+1})/((1-\alpha)(kc+1))$, with draft/target cost ratio $c \approx 1$, is below one for any acceptance rate. Our measured mean accepted length per round (5.8 tokens for the fine-tuned draft at k=8 on natural output, 2.3–2.5 for the un-tuned draft and for forced output) confirms that verification works as designed—the target needs well under one forward pass per emitted token (0.31 for forced output at k=8)—while draft cost cancels the gain. A faster configuration would need a much shallower draft (fewer layers) or a larger target. Memory is not an issue (extra allocation growth about 240 MB, similar to the plain cache).

I. Configuration B: CPU Laptop Model

Configuration B repeats the experiment on the laptop CPU with SmolLM2-360M as target (8.68M LoRA parameters, 2 epochs, 3.6 h of training) and SmolLM2-135M as draft. It matters for two reasons: it is the model the locally hosted application loads by default, and it shows how much of Configuration A's behaviour depends on model scale.

Fine-tuning and before/after. Table XVI and Fig. 11 show that after epoch 1 the model's validation accuracy (0.621) and macro-F1 (0.215) are *identical* to the majority-label baseline: it had learned the answer format and the common label, nothing more. After epoch 2 (selected, validation macro-F1 0.317) it has started to discriminate inclusion classes. Table XVII gives the before/after comparison on identical rows. The zero-shot 360M base model reaches test accuracy 0.437 and macro-F1 0.234 (it follows the schema, 0 invalid outputs, but cites no evidence at all: evidence F1 0.000); after fine-tuning, accuracy is 0.626 (interval 0.472–0.748), macro-F1 0.333 (interval 0.237–0.408) and micro evidence F1 0.123 (interval 0.024–0.259). **This is the same accuracy as the majority baseline** (0.626), and the macro-F1 interval contains the baseline's 0.272; with 8 test patients we therefore cannot claim a reliable improvement over trivial prediction for this configuration, only a point-estimate gain in macro-F1 (0.333 vs. 0.272) driven by inclusion criteria. Validation shows the same picture (0.684 accuracy and 0.317 macro-F1 against 0.621 and 0.215 for the majority label).

Per-class behaviour. For exclusion criteria the fine-tuned model answers *not excluded* on all 131 rows, i.e. it reproduces the majority label exactly (exclusion accuracy 0.695, macro-

TABLE XII

CONFIG. A KV-CACHE BENCHMARK (MEDIAN OF 3 REPETITIONS AFTER WARM-UP; PEAK Δ IS THE CUDA ALLOCATOR PEAK ABOVE THE PRE-CALL LEVEL, KV = CACHE BYTES IN USE; THE LAST COLUMN CHECKS TOKEN EQUALITY WITH THE UNCACHED REFERENCE).

In	Out	Mode	Prefill (s)	Decode (s)	tok/s	Peak Δ (MB)	KV (MB)	= ref.
64	16	uncached	0.102	2.325	6.6	14	0.0	yes
		kv_cache	0.100	0.491	26.8	107	29.6	yes
		kv_cache_tiled	0.094	0.884	16.4	107	29.6	yes
64	64	uncached	0.125	12.331	5.1	22	0.0	yes
		kv_cache	0.089	2.024	30.2	107	47.6	yes
		kv_cache_tiled	0.090	3.611	17.3	107	47.6	yes
64	128	uncached	0.112	29.406	4.3	33	0.0	yes
		kv_cache	0.102	4.153	30.1	107	71.6	yes
		kv_cache_tiled	0.105	8.277	15.3	107	71.6	yes
256	16	uncached	0.327	6.490	2.3	47	0.0	yes
		kv_cache	0.334	0.511	18.9	149	101.6	yes
		kv_cache_tiled	0.335	1.617	8.2	149	101.6	yes
256	64	uncached	0.353	28.947	2.2	55	0.0	yes
		kv_cache	0.328	2.115	26.2	167	119.6	yes
		kv_cache_tiled	0.351	6.659	9.1	167	119.6	yes
256	128	uncached	0.347	60.917	2.1	66	0.0	yes
		kv_cache	0.328	4.308	27.6	191	143.6	yes
		kv_cache_tiled	0.353	14.498	8.6	191	143.6	yes
512	16	uncached	0.683	12.507	1.2	91	0.0	yes
		kv_cache	0.589	0.528	14.3	289	197.6	yes
		kv_cache_tiled	0.669	2.623	4.9	289	197.6	yes
512	64	uncached	0.656	49.965	1.3	99	0.0	yes
		kv_cache	0.612	2.217	22.6	307	215.6	yes
		kv_cache_tiled	0.697	11.065	5.5	307	215.6	yes
512	128	uncached	0.645	95.949	1.3	110	0.0	yes
		kv_cache	0.616	4.543	24.8	331	239.6	yes
		kv_cache_tiled	0.714	22.456	5.5	331	239.6	yes

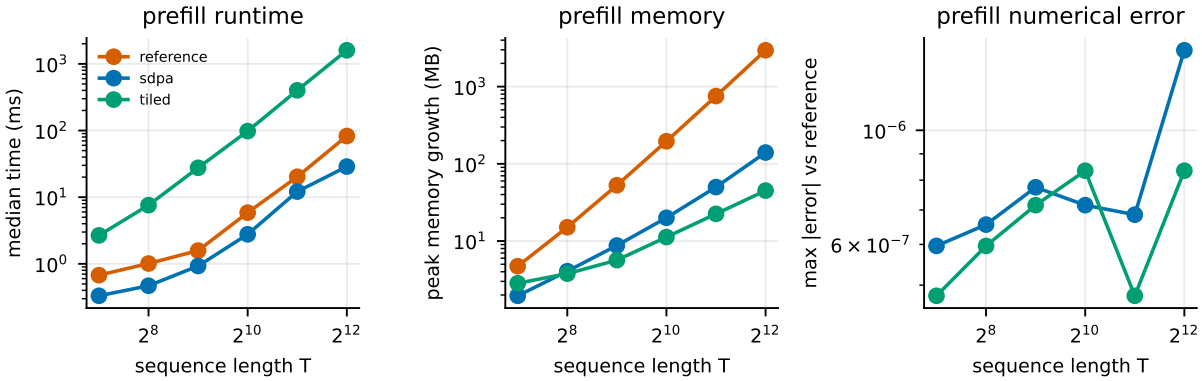


Fig. 8. Config. A prefill attention: runtime, memory and error versus sequence length (log scales).

F1 0.273), and for inclusion criteria it answers *not enough information* 57 times and *included* 18 times (Table XVIII, Fig. 12): *included* reaches only P 0.44, R 0.30, F1 0.36 against 0.79, 0.70 and 0.75 for the 1.7B model, and *not included* and *not applicable* are never predicted. *Not enough information* for inclusion criteria is its best class (F1 0.63).

Evidence. Micro precision is 0.350 but recall only 0.074 (inclusion: P 0.350, R 0.113, F1 0.171; exclusion: nothing cited); on evidence-bearing records the macro F1 is 0.109, whereas the all-record macro F1 of 0.707 merely reflects that the model leaves 0.959 of the empty-gold records empty (6 spurious citations).

Errors. Of 206 test rows 101 are correct in label and evidence, 50 are false decisions (gold *not enough information* 36 or *not applicable* 14, answered *not excluded* 40 times and *included* 10 times), 28 have the right label with different evidence, 26 are missed decisions (a decisive gold label answered *not enough information* or *not applicable*) and 1 is a *not applicable* confusion (Table XIX); 131 predictions carry a decisive label without any cited sentence, 115 of which also have empty expert evidence and 16 of which do not. The twelve-case listing for Configuration A (Table XI)

TABLE XIII
CONFIG. A ATTENTION BENCHMARK: PREFILL (T_q=T_k=T) AND DECODE (T_q=1) FOR THREE LENGTHS; TILED USES 64×64 TILES; ERRORS ARE VERSUS THE REFERENCE.

Phase	Impl.	T	Time (ms)	Max err.	Mean err.	Peak Δ (MB)
prefill	reference	256	1.01	0.0e+00	0.0e+00	15
prefill	sdpa	256	0.47	6.6e-07	3.1e-08	4
prefill	tiled	256	7.58	6.0e-07	2.7e-08	4
prefill	reference	1024	5.86	0.0e+00	0.0e+00	196
prefill	sdpa	1024	2.77	7.2e-07	3.0e-08	20
prefill	tiled	1024	98.06	8.3e-07	2.3e-08	11
prefill	reference	2048	20.25	0.0e+00	0.0e+00	754
prefill	sdpa	2048	12.13	6.9e-07	2.9e-08	50
prefill	tiled	2048	401.02	4.8e-07	2.2e-08	22
decode	reference	256	0.67	0.0e+00	0.0e+00	2
decode	sdpa	256	0.36	1.5e-07	2.9e-08	2
decode	tiled	256	3.09	7.5e-08	1.1e-08	1
decode	reference	1024	0.66	0.0e+00	0.0e+00	8
decode	sdpa	1024	0.82	1.6e-07	2.6e-08	8
decode	tiled	1024	12.01	6.0e-08	9.5e-09	1
decode	reference	2048	0.65	0.0e+00	0.0e+00	15
decode	sdpa	2048	1.04	1.7e-07	2.4e-08	15
decode	tiled	2048	25.14	6.0e-08	9.2e-09	1

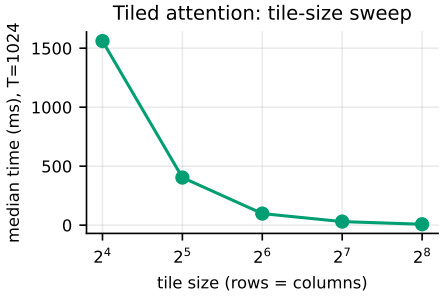


Fig. 9. Config. A: effect of tile size on tiled-attention time at $T = 1024$.

illustrates the same failure types; the CPU cases are in `results/error_analysis_cpu_finetuned_test.md`. The comparison with Configuration A suggests that, on this data, a 1.7B model with three epochs begins to learn inclusion reasoning (included recall 0.70 vs. 0.30, evidence micro-F1 0.384 vs. 0.123) whereas a 360M model with two epochs mostly learns the label prior; both collapse on exclusion criteria.

Draft model (SmolLM2-135M). Table XX shows the draft's validation behaviour over three epochs (2.3 h on the CPU). Validation loss keeps falling (0.220, 0.194, 0.191), but the selection rule (highest validation macro-F1) picks epoch 1 (0.267) because macro-F1 is higher there than at epochs 2 and 3 (both 0.241); the differences are a few points on 206 rows and are not meaningful beyond showing that the 135M model learns the answer format and the label prior and little else. This is acceptable for a draft, whose purpose is to propose the target's likely tokens, not to be accurate by itself, but it means that the acceptance rates of the CPU pair were not benchmarked in a stored table (see below) and should not be inferred from the GPU pair.

TABLE XIV
CONFIG. A SPECULATIVE DECODING, NATURAL TASK OUTPUT (32 PROMPTS; EQUAL = OUTPUTS IDENTICAL TO GREEDY TARGET; ACCEPT. = ACCEPTED / PROPOSED DRAFT TOKENS).

Draft	k	Equal	Accept.	Latency (s)	tok/s	Speed-up
greedy target		–	–	0.894	15.0	1.00
finetuned draft	2	32/32	97%	1.180	11.4	0.76
finetuned draft	4	32/32	98%	1.133	11.8	0.79
finetuned draft	6	32/32	96%	1.102	12.2	0.81
finetuned draft	8	32/32	92%	1.110	12.1	0.81
untuned draft	2	32/32	64%	1.340	10.0	0.67
untuned draft	4	32/32	58%	1.370	9.8	0.65
untuned draft	6	32/32	49%	1.480	9.1	0.60
untuned draft	8	32/32	40%	1.630	8.2	0.55

TABLE XV
CONFIG. A SPECULATIVE DECODING, FORCED 64-TOKEN OUTPUT.

Draft	k	Equal	Accept.	Latency (s)	tok/s	Speed-up
greedy target		–	–	2.571	24.9	1.00
finetuned draft	2	32/32	65%	4.230	15.1	0.61
finetuned draft	4	32/32	49%	4.798	13.3	0.54
finetuned draft	6	32/32	39%	5.743	11.1	0.45
finetuned draft	8	32/32	33%	6.547	9.8	0.39
untuned draft	2	32/32	63%	4.304	14.9	0.60
untuned draft	4	32/32	52%	4.629	13.8	0.56
untuned draft	6	32/32	41%	5.543	11.5	0.46
untuned draft	8	32/32	33%	6.471	9.9	0.40

What the laptop is used for. Configuration B provides the model of the locally hosted application, whose *Compare decoding* tab measures all four decoding modes live on the user's CPU (Section V-J); we did not run the full stored benchmark grid of Section V-F–V-H on the CPU, because the GPU grid already covers prompt lengths, output lengths, attention shapes and draft settings with a larger, more realistic model and any CPU table would add timings but no new correctness evidence. CPU-specific numbers should therefore be taken from the live tab rather than from this paper.

J. Application

The application is a self-contained web service: `python app/server.py` (or `run_app.bat`) starts a FastAPI server on `127.0.0.1:8000`, loads the target and draft models and merges the LoRA adapters in a background thread, and serves a single-page browser interface; no notebook, command-line step or IDE is needed afterwards. The header shows a status indicator (starting, ready, error), the loaded model and adapter, and a persistent banner states that this is an educational screening prototype and not a clinical enrollment decision. Loading takes well under a minute on the laptop; requests that arrive earlier receive a clear "model not ready" message instead of an error. If an adapter is missing, the base model is used and the header says so.

Screening workflow (Fig. 13). A user supplies a patient note by pasting text or loading one of the held-out sample patients, and a trial by choosing a sample trial, fetching one from ClinicalTrials.gov by NCT identifier (public metadata

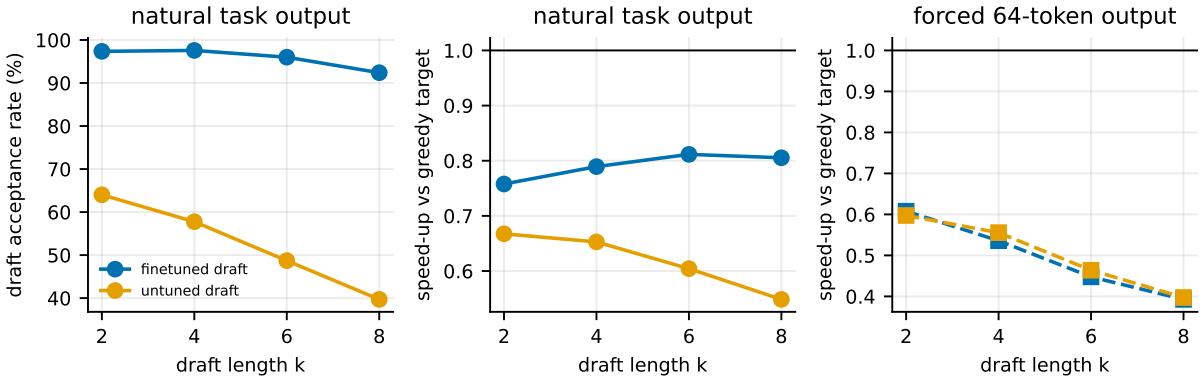


Fig. 10. Config. A: acceptance rate and speed-up against draft length k for the fine-tuned and un-tuned draft.

TABLE XVI
CONFIG. B TARGET (SMOLLM2-360M + LORA, CPU): VALIDATION AFTER EACH EPOCH (* = SELECTED).

Epoch	Val. loss	Val. acc.	Val. macro-F1	Val. evid. F1
1	0.207	0.621	0.215	0.214
2*	0.189	0.684	0.317	0.196

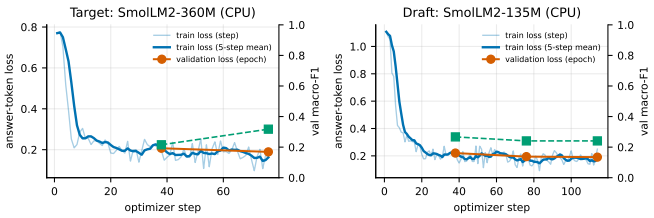


Fig. 11. Config. B training curves (CPU): target 360M and draft 135M.

only), pasting an "Inclusion Criteria / Exclusion Criteria" block that is split into individual criteria, or typing criteria manually. Notes that are not already numbered are split into sentences and numbered from zero, with a preview of the numbering. For every criterion the interface shows the predicted label, the cited sentences quoted from the note, an explanation that only restates the label and the cited text, the raw model output, latency and generated tokens, and, for the sample patients, the dataset's expert label with a match indicator; selecting a criterion highlights its cited sentences in the numbered note (yellow) and the expert's sentences (green marker). The preliminary outcome (here "Needs more information") is computed by the explicit rule of Section III-H and is displayed together with that rule. Missing information is never presented as eligibility: a criterion with no cited sentence is shown with the text "No supporting sentence cited".

Report export and provenance. The *Download report* buttons produce a readable HTML report (also suitable for printing to PDF), a Markdown file or a JSON file containing the disclaimer, the trial, the preliminary outcome with its rule, every criterion with label, evidence text and explanation, the numbered note, and provenance: generation time, model, a SHA-256 prefix of the adapter file, the decoding settings and a hash of the note.

Input validation and recovery (Fig. 14). The server validates every request: notes shorter than 20 characters, unknown criterion types, criteria shorter than 3 characters or empty reports are rejected with an explanatory message and HTTP 422; an unreachable registry yields a message asking the user to paste the criteria instead; prompts above 3 000 tokens are refused rather than silently truncated; and if one criterion fails during a multi-criterion run it is reported as invalid while the others continue. After an error the user corrects the field and runs again without reloading.

Inference comparison (Fig. 15). The *Compare decoding* tab runs the current note and a chosen criterion through the four implemented modes—uncached reference, KV cache, KV cache with tiled attention, and speculative decoding with a selectable draft length—with a warm-up pass and selectable repetitions, optionally forcing a longer output. It reports whether each mode's tokens are identical to the first, prefill, decode and total time, tokens per second, speed-up, memory growth, target forward passes and, for speculative decoding, the acceptance rate, and draws latency and throughput bars. These numbers are measured live on the machine hosting the application, not read from images; the *Experiments* tab additionally renders the stored result files (before/after metrics, KV, attention and speculative benchmarks). In the screenshot, taken on the idle laptop CPU with the 360M target and the tuned 135M draft (231 prompt tokens, 14 generated tokens, one repetition), all four modes return identical tokens; the KV cache cuts end-to-end latency from 13.97 s to 2.76 s (5.1×), our tiled kernel is slower than SDPA inside the decoder (3.56 s), and speculative decoding accepts 79% of drafted tokens at k=4 and needs only 5 target passes instead of 14, but at 3.00 s it is still slightly slower than plain cached decoding because each draft pass is not much cheaper than a target pass on a CPU. This single live measurement is illustrative, not a benchmark.

TABLE XVII
CONFIG. B (CPU): MAJORITY BASELINE, BASE ZERO-SHOT AND FINE-TUNED SMOLLM2-360M ON THE SAME ROWS.

Split	Model	Acc.	mF1	Incl. acc.	Incl. mF1	Excl. acc.	Excl. mF1	Invalid
Val.	majority label	0.621	0.215	0.409	0.145	0.721	0.210	0
	base zero-shot	0.495	0.229	0.136	0.113	0.664	0.276	0
	fine-tuned	0.684	0.317	0.606	0.310	0.721	0.210	0
Test	majority label	0.626	0.272	0.507	0.168	0.695	0.273	0
	base zero-shot	0.437	0.234	0.120	0.070	0.618	0.350	0
	fine-tuned	0.626	0.333	0.507	0.247	0.695	0.273	0

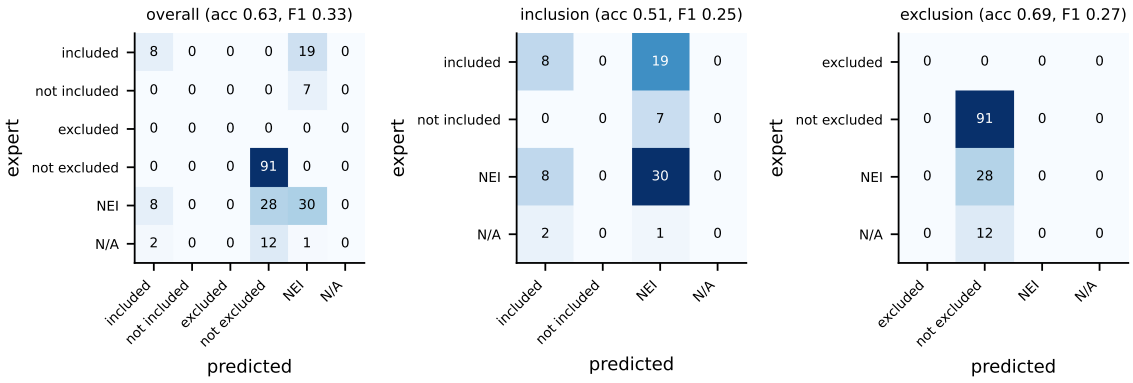


Fig. 12. Config. B confusion matrices on the test split.

TABLE XVIII
CONFIG. B FINE-TUNED MODEL, TEST SPLIT: PER-CLASS P, R, F1 AND SUPPORT.

Scope	Class	P	R	F1	Support
inclusion	included	0.444	0.296	0.356	27
	not included	0.000	0.000	0.000	7
	not enough info.	0.526	0.789	0.632	38
	not applicable	0.000	0.000	0.000	3
exclusion	excluded	0.000	0.000	0.000	0
	not excluded	0.695	1.000	0.820	91
	not enough info.	0.000	0.000	0.000	28
	not applicable	0.000	0.000	0.000	12

TABLE XIX
CONFIG. B TEST ERRORS BY CATEGORY.

Error category	Count
correct	101
false decision	50
evidence only	28
missed decision	26
na confusion	1

VI. LIMITATIONS

Small, clustered test set. The test split has 8 patients and 206 criteria that are not independent; differences of a few points are within the bootstrap intervals, and rare classes (*not included*, *excluded*) have a handful of examples or none. Conclusions about those classes are therefore weak.

TABLE XX
CONFIG. B DRAFT (SMOLLM2-135M + LORA, CPU): VALIDATION AFTER EACH EPOCH (* = SELECTED).

Epoch	Val. loss	Val. acc.	Val. macro-F1	Val. evid. F1
1*	0.220	0.636	0.267	0.175
2	0.194	0.631	0.241	0.048
3	0.191	0.631	0.241	0.254

Model scale and hardware. Models of 0.36–1.7B parameters trained for two to four epochs on 602 examples are far from the capability of GPT-4-class systems; we did not tune LoRA rank, learning rate or epoch count beyond the single configuration per model reported, and we did not run multiple seeds, so run-to-run variance is unknown. Epoch selection used validation macro-F1, which moves by several points between epochs and is itself noisy. The GPU numbers come from a shared Kaggle T4 whose clock and neighbours we do not control.

Shortness and style of notes. The corpus contains short synthetic descriptions; real electronic health records are longer, noisier and would need chunking and retrieval before this prompt format applies. The app rejects prompts above 3 000 tokens rather than truncating silently.

Explanations are extractive. They restate the label and the cited sentences; they are not free-text clinical reasoning and cannot detect an unsupported citation beyond checking that

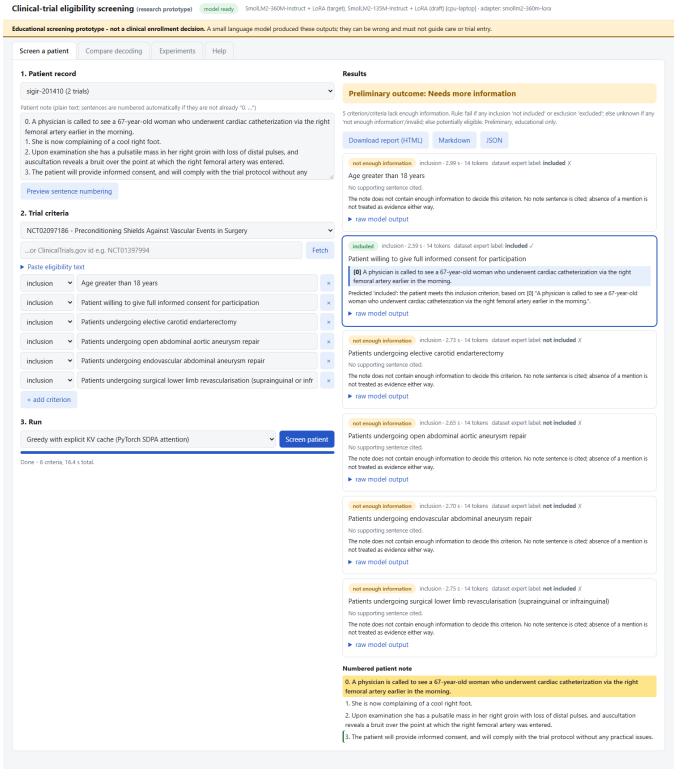


Fig. 13. Screening workflow: sample patient and trial, six criteria screened, per-criterion label and cited evidence, preliminary outcome, report export buttons, and the numbered note with the selected criterion's evidence highlighted.

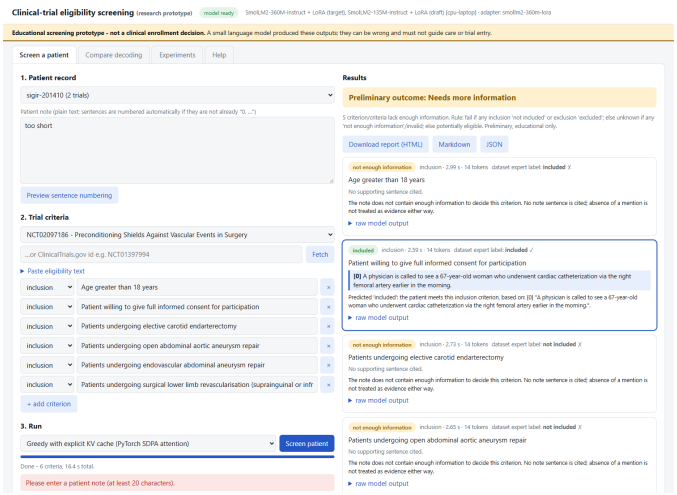


Fig. 14. Recovery from an invalid input: a too-short note is rejected with an explanatory message and the form remains usable.

the ID exists in the note.

No patient-level ground truth. The preliminary outcome is a transparent rule over criterion labels; it has not been validated against any patient-trial label and should not be read as one.

Benchmark validity. Stored benchmark timings come from a single shared Kaggle T4 session, so clocks and neighbouring load are not controlled; the CPU configuration has no stored

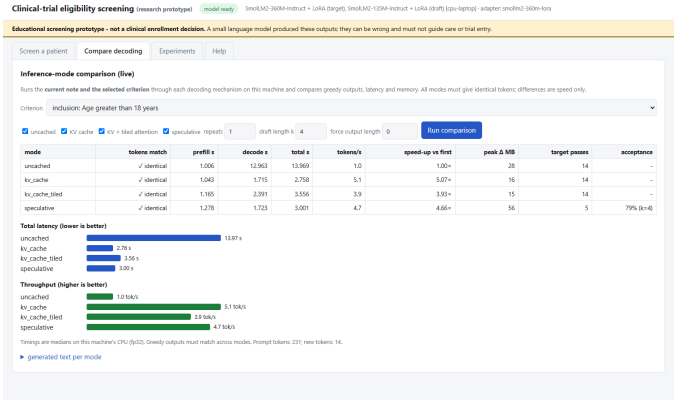


Fig. 15. *Compare decoding* tab: live comparison of the four decoding modes on the same input, measured on the idle laptop CPU (Configuration B).

benchmark tables, only the live comparison in the application. The educational tiled attention is pure PyTorch and says nothing about fused GPU kernels; the speculative-decoding slowdown measured here depends on the layer-count ratio of this particular draft/target pair (32 vs. 24 layers) and on the very short natural outputs, and a different pair could behave differently.

Safety. The system is an educational screening prototype. It must not be used for enrollment, treatment or any other clinical decision.

VII. Conclusion

We built, trained and evaluated an end-to-end criterion-level eligibility screener on the TrialGPT annotation corpus with a strictly patient-grouped split, and implemented the Transformer inference mechanisms ourselves rather than through a library switch. Fine-tuning SmolLM2-1.7B with LoRA lifts test macro-F1 from 0.111 (zero-shot) to 0.460 and accuracy from 0.146 to 0.709, with evidence F1 0.384; however, the accuracy gain over the 0.626 majority baseline is not statistically established, the improvement is confined to inclusion criteria, and for exclusion criteria the model almost always answers *not excluded*, treating missing information as a negative finding in 40 of 42 such cases. The explicit KV cache reproduces uncached greedy output exactly and is 18.7× faster at 512+128 tokens at the price of a cache that grows by 393 kB per token for the 1.7B model; the tiled online-softmax attention matches the reference to 10^{-6} and bounds attention memory linearly in sequence length, but is much slower than fused kernels in pure PyTorch; and our greedy speculative decoder is token-exact against the target and against Hugging Face assisted generation, but with a draft that is as deep as the target it is slower than plain decoding on a GPU, which we explain with the cost ratio. Future work should address the exclusion-criterion collapse (class-balanced or cost-sensitive training, an explicit abstention head, or retrieval of relevant sentences), evaluate several seeds, and pair the target with a shallower draft. The system is an educational prototype and not a tool for clinical decisions.

Code, Data and Model Availability

Source code, configuration files, the split manifest, result files, benchmark logs and reproduction instructions are available at https://github.com/Zaid-fareed/i230829_C_GenAi-A2. The trained LoRA adapters (SmolLM2-1.7B and 360M from the GPU configuration; 360M and 135M from the CPU configuration) are hosted at https://huggingface.co/ZaidFareed/i230829-genai-a2-adapters and can be installed with `python scripts/download_adapters.py ZaidFareed/i230829-genai-a2-adapters`. The base models are downloaded from the Hugging Face Hub by `scripts/download_models.py`, and the dataset is the public TrialGPT Criterion Annotations release [3]. The application demonstration video is available at https://youtu.be/uVTm5AuokaU.

Acknowledgment and AI-Use Statement

Claude (Anthropic), used through the Claude Code assistant, supported this work as a coding and writing assistant. It produced first drafts of the source code, scripts and browser application and a first draft of this report, and it helped diagnose and fix errors during development. The author defined the requirements from the assignment brief, provided the inputs and direction for each step, and chose between the alternatives that were considered (for example, moving the main experiments to a free Kaggle GPU, the SmolLM2 model pair, and which experiments to run or to skip). The author ran the GPU fine-tuning, evaluation and benchmarks on Kaggle and resolved the errors that occurred there with the assistant's help; the laptop (CPU) experiments were run locally with the assistant. The author tested the browser application, reviewed the code and the reported results, and reviewed and revised this report. Reused libraries and pre-trained weights (PyTorch, Hugging Face Transformers and PEFT, FastAPI, SmolLM2, Qwen2.5) are cited or named in the text and were tested rather than assumed correct: the re-implemented decoder was checked against the Hugging Face logits, the cache, tiled attention and speculative decoder against reference paths, and all tables are generated by scripts from saved result files. No code or results were taken from the TrialGPT repository; only its public annotation dataset was used. The author is responsible for the submitted work.

References

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, p. 9074, 2024.

[2] B. Koopman and G. Zuccon, "A test collection for matching patients to clinical trials," in *Proc. 39th Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2016.

[3] NCBI NLP, "TrialGPT-Criterion-Annotations dataset," Hugging Face Datasets, https://huggingface.co/datasets/ncbi/TrialGPT-Criterion-Annotations, 2024, accessed 2026.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Int. Conf. on Learning Representations (ICLR)*, 2022.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[6] L. B. Allal, A. Lozhkov, E. Bakouch *et al.*, "SmolLM2: When smol goes big – data-centric training of a small language model," *arXiv preprint arXiv:2502.02737*, 2025.

[7] H. Touvron, T. Lavril, G. Izacard *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[8] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[9] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[10] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[11] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[12] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[14] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[15] M. N. Rabe and C. Staats, "Self-attention does not need $O(n^2)$ memory," *arXiv preprint arXiv:2112.05682*, 2021.

[16] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[17] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proc. Machine Learning and Systems (MLSys)*, 2023.

[18] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symp. on Operating Systems Principles (SOSP)*, 2023.

[19] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.

[20] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. 40th Int. Conf. on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 19 274–19 286.

[21] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[22] T. Wolf, L. Debut, V. Sanh *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020.